Explainable Code Change Risk Predictor

Spotting the commits most likely to introduce a bug, and saying why.

Akshat Sparsh · 23BDS0149
Guided by Dr. Vijayanand C

Newest commits to psf/requests, scored by the model

85400d8	Allow use of 'no_proxy' in proxies	76% High
7fd9267	Remove final remnants from 2.6	76% High
58cea7a	Drop support for CPython 3.7	47% Medium
2e1086f	Adding comment	47% Medium
e5894e9	Improve flake8 help.py	21% Low
1f6de03	Cleanup certs.py	19% Low

Review time is limited

In most teams a one-line typo fix and a 600-line refactor get roughly the same attention in code review.

Bugs slip through in the big, scattered changes, and by the time they show up weeks later they cost far more to fix.

Our goal: a model that ranks commits by risk the moment they are made, and gives reasons a developer can act on.

`commit e5894e9` 21% Low

Improve flake8 help.py

1 file · 0 lines added · 1 deleted

`commit b7809ac` 77% High

Making class and function docstrings consistent

9 files · 101 lines added · 92 deleted

Two real commits from the test period. Which one would you review first?

What the tool does

1. Takes any commit from a git repository
2. Computes 14 measurements about it, like its size, its spread and its author's experience
3. Predicts the chance it introduces a bug
4. Explains the score in plain English, using SHAP

```
$ uv run python -m riskpredictor.explain --commit 85400d8

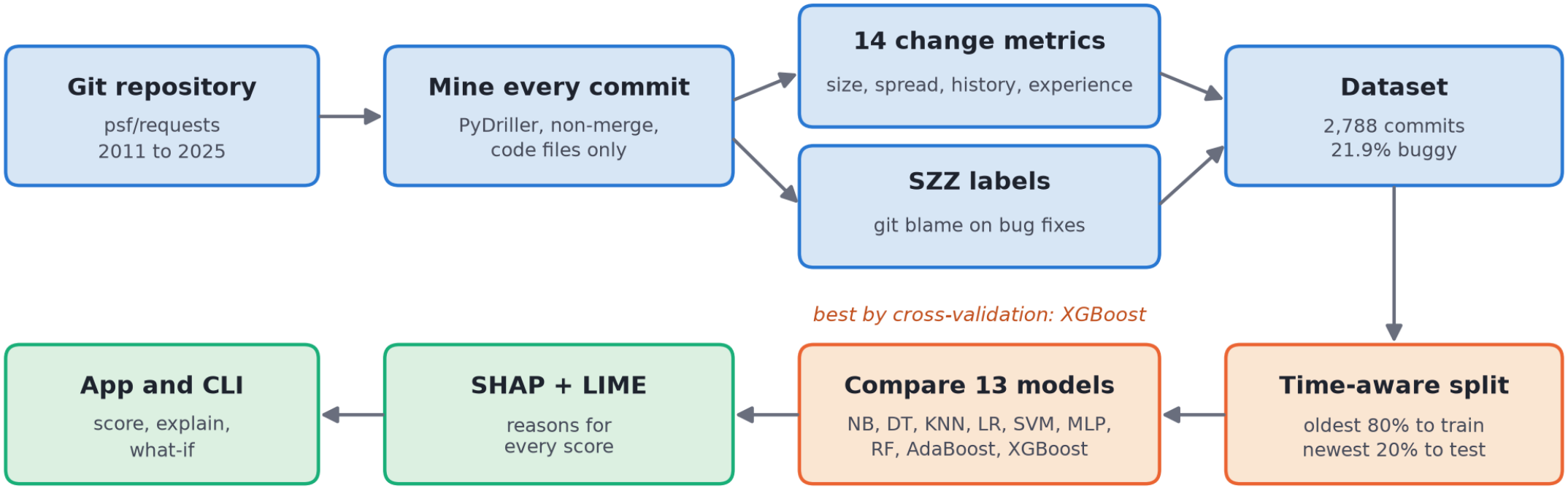
Commit 85400d8d: Allow use of 'no_proxy' in the proxies argument

Predicted risk : 76.1%  (High risk, FLAGGED)
Average commit : 44.8%

Factors raising the risk:
  1. Adds 91 lines, more than 96% of past commits,
     which raises the risk by 18.0 percentage points.
  2. Modifies 3 files, more than 94% of past commits,
     which raises the risk by 3.9 percentage points.
  3. Spreads its changes across files with entropy 0.88,
     which raises the risk by 2.4 percentage points.

Actual label: buggy (a later fix was traced back to it)
```

How it works



Everything runs from one Python package. Each step is a single command and is covered by tests.

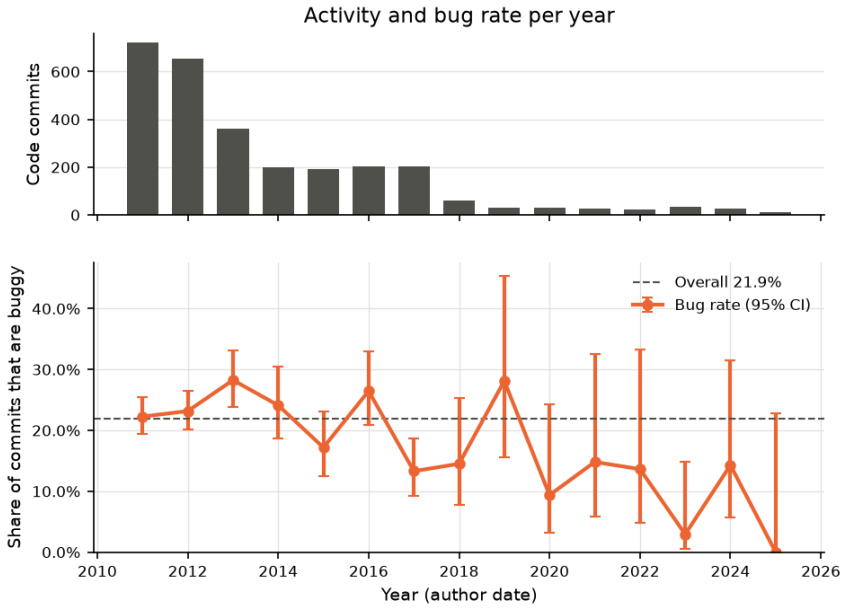
The data: 14 years of psf/requests

2,788
commits that change code, mined with PyDriller

21.9%
of them later turned out to introduce a bug

14
features per commit, from Kamei et al. (2013)

2011
to Aug 2025, the full project history



Why requests? It is one of the most used Python libraries, with a long and public history of bugs and fixes.

What we kept: non-merge commits that touch code. We dropped the last 180 days because their bugs mostly haven't been found yet.

Worth noticing: most activity was in the early years, and the bug rate drifts over time. That is why we split the data by date.

14 features in five groups

D

Diffusion

How spread out is the change?

`ns, nd, nf, entropy`

S

Size

How big is it?

`la, ld, lt`

P

Purpose

Is it fixing a bug?

`fix`

H

History

How busy are the files it touches?

`ndev, age, nuc`

E

Experience

How experienced is the author?

`exp, rexp, sexp`

A real example: commit `85400d8`, "Allow use of 'no_proxy' in the proxies argument"

`ns` 2 · `nd` 2	`la` 91 · `ld` 15	`fix` 0	`ndev` 130 · `nuc` 482	`exp` 0 · `rexp` 0
`nf` 3 · `entropy` 0.88	`lt` 702		`age` 0 days	`sexp` 0 (first commit)

History and experience are computed only from earlier commits. We walk the history from oldest to newest, so the model can never peek into the future.

Where the labels come from: SZZ

1

Find the fixes

Commit messages that say `fix`, `bug` or `defect`. We found 518.

2

Blame the lines

Run `git blame` on every line each fix changed.

3

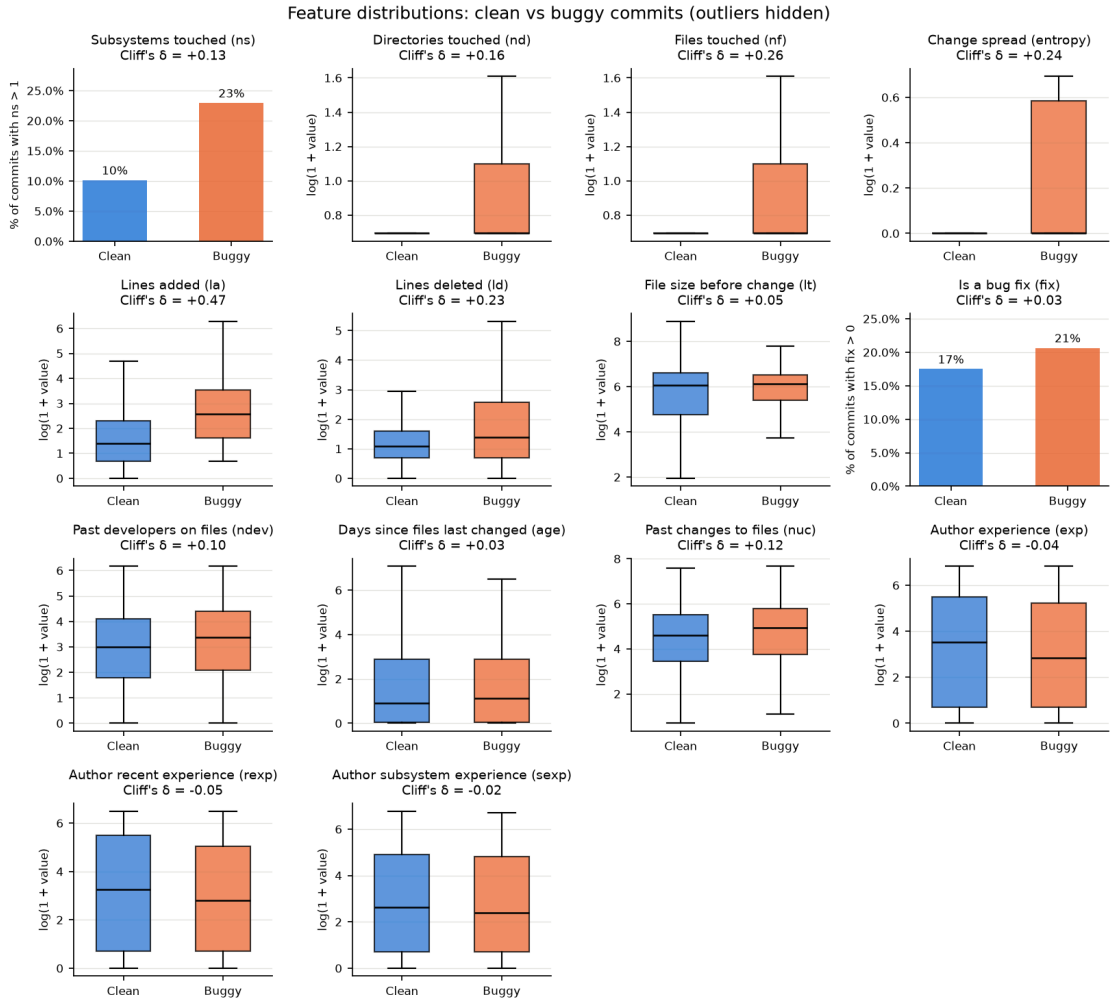
Label the culprit

The commit that last touched those lines is labelled buggy. That gave 613 commits.

It isn't perfect

Keywords miss some fixes. Fixes that only add lines can't be traced. A commit that only moved buggy code can get the blame. And very recent commits look clean because nobody has found their bugs yet.

What separates buggy commits?



12 vs 3

median lines added by buggy vs clean commits, the biggest difference of all

8 of 14 features differ significantly (Mann-Whitney U test, $p < 0.05$). Size and spread matter most.

Author experience and the fix flag barely differ on their own.

Every distribution overlaps a lot, so no single feature is enough. We need a model that combines them.

Getting the data ready

What kind of data

Most features are counts on a ratio scale. The label is nominal, the date is interval, and size bands are ordinal.

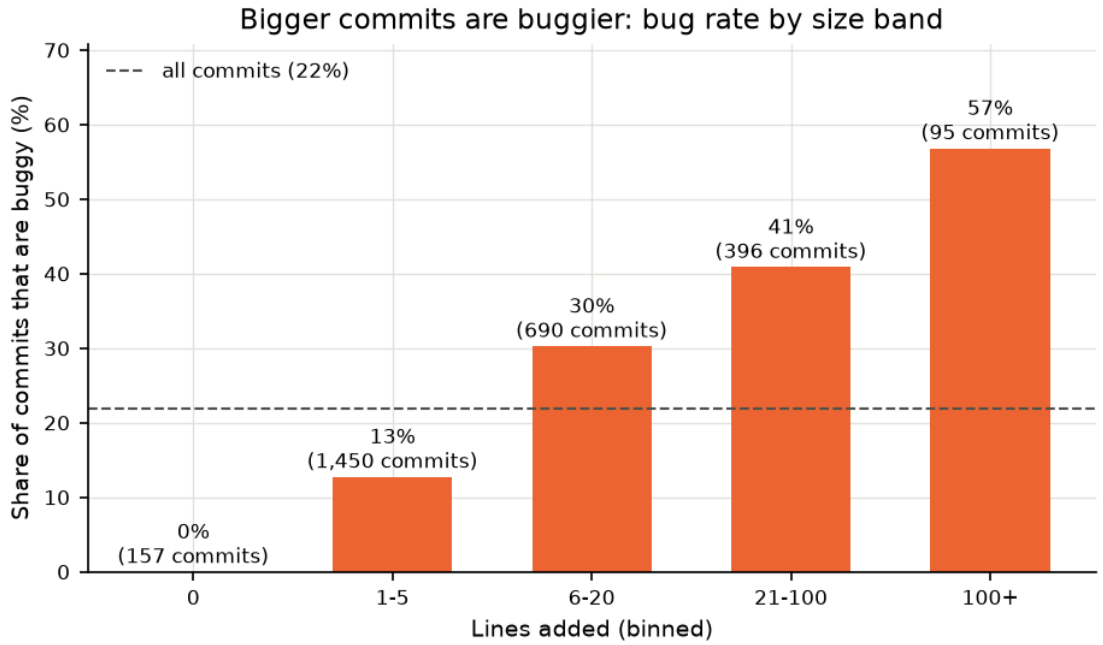
Duplicates

No repeated ids. 11 commits were the same change saved twice; removing them moves the bug rate from 21.88% to 21.97%.

9×

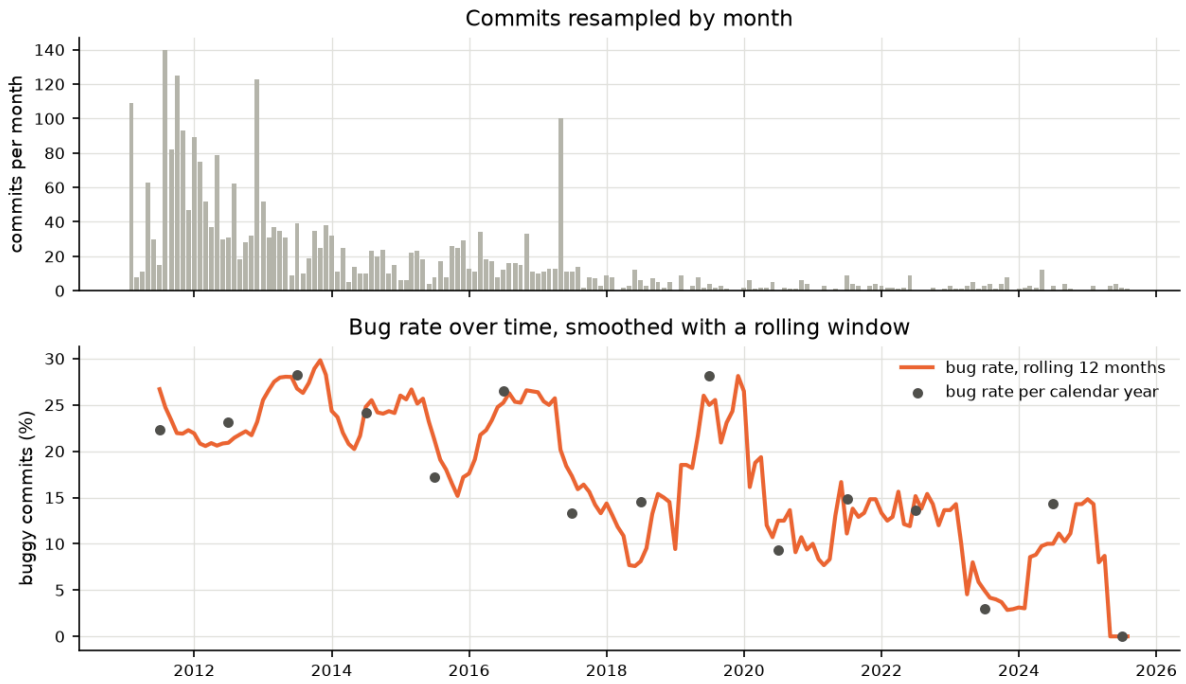
Missing values

Filling gaps by maximum likelihood (EM) was about 9 times more accurate than using the mean.



Binning lines added into bands shows the bug rate climbing from 13% to 57% as commits get bigger.

Over time, and in tables



Resampled by month and smoothed with a 12-month rolling window. The bug rate drifts down: 23.9% up to 2014, 17.3% after.

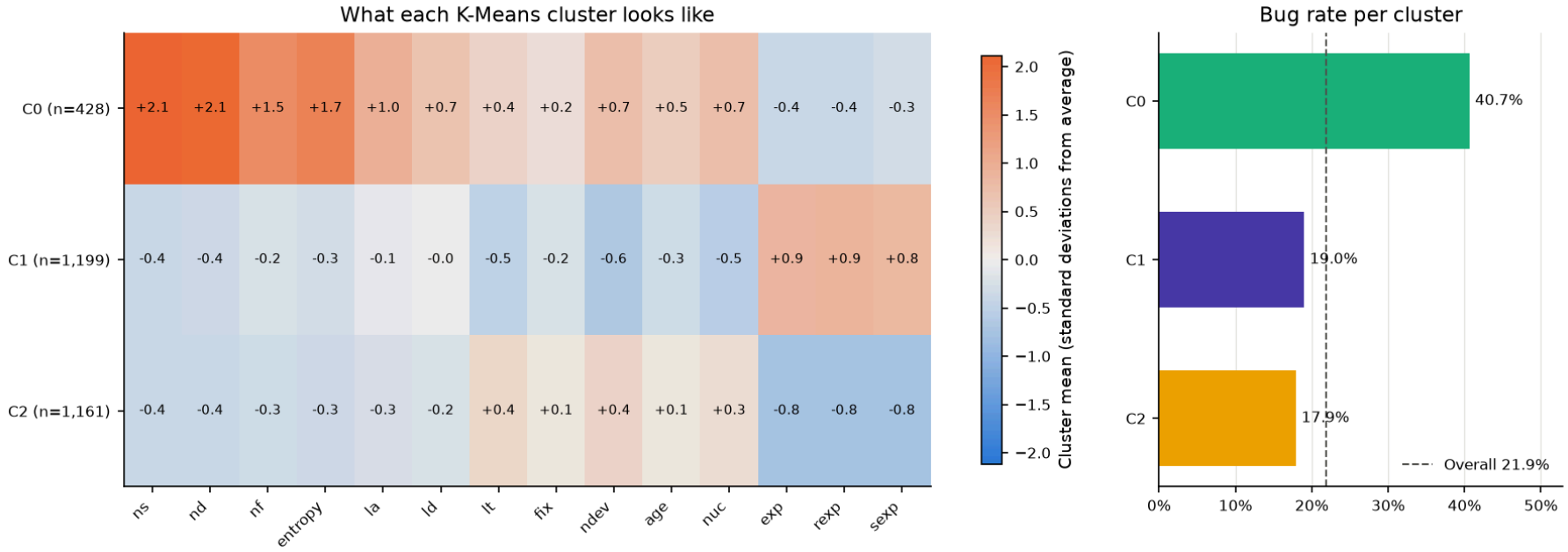
Contingency tables with chi-square

Does it matter?	p-value	Verdict
Size band	< 0.001	**yes**
Bug fix or not	0.084	not at 5%
Weekday	0.101	not at 5%

The Bayesian view

There's a 96% chance bug-fix commits are riskier, and 95% for Friday commits. The evidence leans that way, but not strongly enough for a classical test.

Clustering finds a risky kind of commit



Cross-cutting changes

Several subsystems and files. **40.7% buggy**

Experienced authors

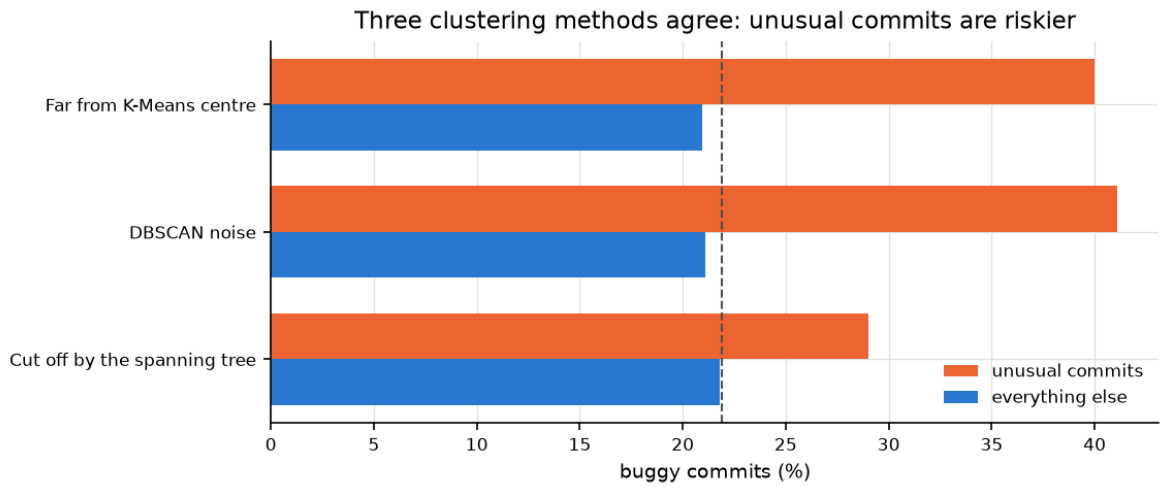
Small changes by regulars. **19.0% buggy**

Newcomers

Small changes by new authors. **17.9% buggy**

K-Means never saw the labels. DBSCAN agreed: the unusual commits it marked as noise were buggy 41% of the time.

More ways to find structure



Three ways of using clustering to find unusual commits. All three say unusual commits are riskier.

Clustering the commit messages

"upgrade urllib3"	**38%**
"fixes issue, redirects"	30%
"remove unused import"	**8%**

Also tried

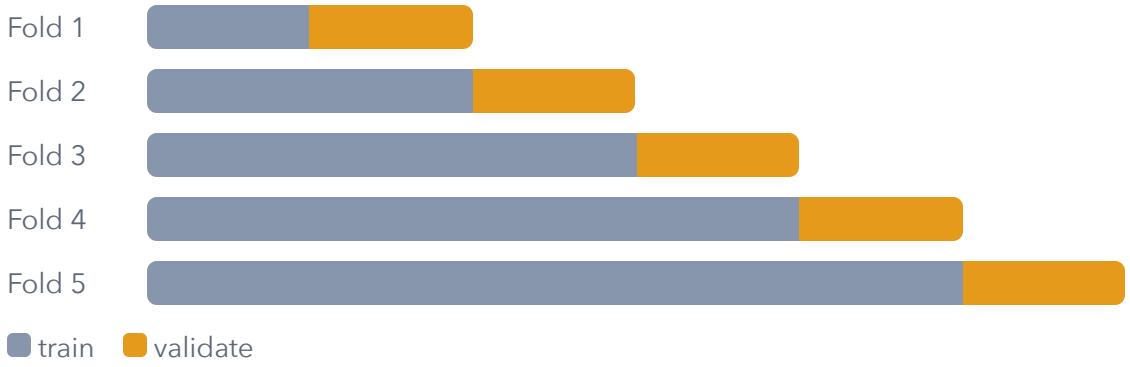
Spectral clustering, Gaussian mixtures (EM), spanning trees, SVD, factor analysis, MDS, Isomap and a self-organizing map. The 14 features really need only about 4 dimensions.

Testing it the honest way

Train: oldest 80%
Feb 2011 to Jun 2016 · 2,230 commits · 23.6% buggy

Test: newest 20%
558 commits · 15.1% buggy

TimeSeriesSplit inside the training period: always train on the past, validate on what comes next



1. **No peeking at the future.** A random split would train on 2020 commits and test on 2012 ones, which can never happen for real.
2. **Tuning stays inside training.** Cross-validation and the hyperparameter search both use these folds.
3. **Test set touched once.** The model and its threshold were chosen without looking at it.

13 algorithms compared

Mean cross-validated ROC-AUC on the training period (how we picked the winner)



A narrow win

The top seven are within one standard deviation of each other. With 14 features and noisy labels, the data sets the ceiling.

Ensembles lead

XGBoost, random forest and AdaBoost all sit near the top. Single trees overfit.

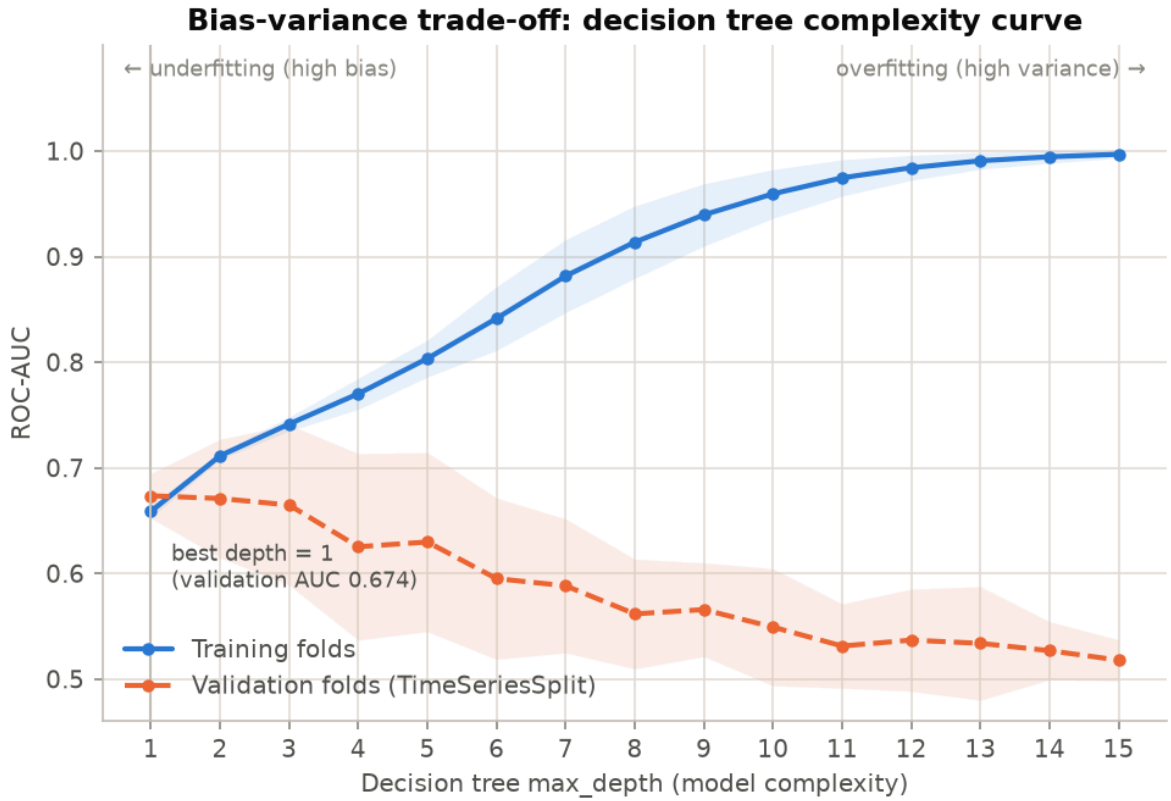
Imbalance and overfitting

Only 22% of commits are buggy

Recall of logistic regression at the default 0.5 threshold:

Handling	CV ROC-AUC	Recall at 0.5
None	0.721	**0.31**
Class weights	0.721	**0.74**
SMOTE	0.719	**0.74**

Rebalancing barely changes the ranking, but without it the model misses about 70% of buggy commits. Either rebalance or tune the threshold.



A depth-15 tree scores **0.997** on its training folds but only **0.518** on validation. That is variance. Ensembles fix it.

Results on the newest commits

0.779

test ROC-AUC for the tuned XGBoost model

0.401

test PR-AUC, where random guessing gets 0.151

80%

of buggy commits caught (67 of 84) at the chosen threshold

48%

of all bug-introducing commits sit in the riskiest 20%

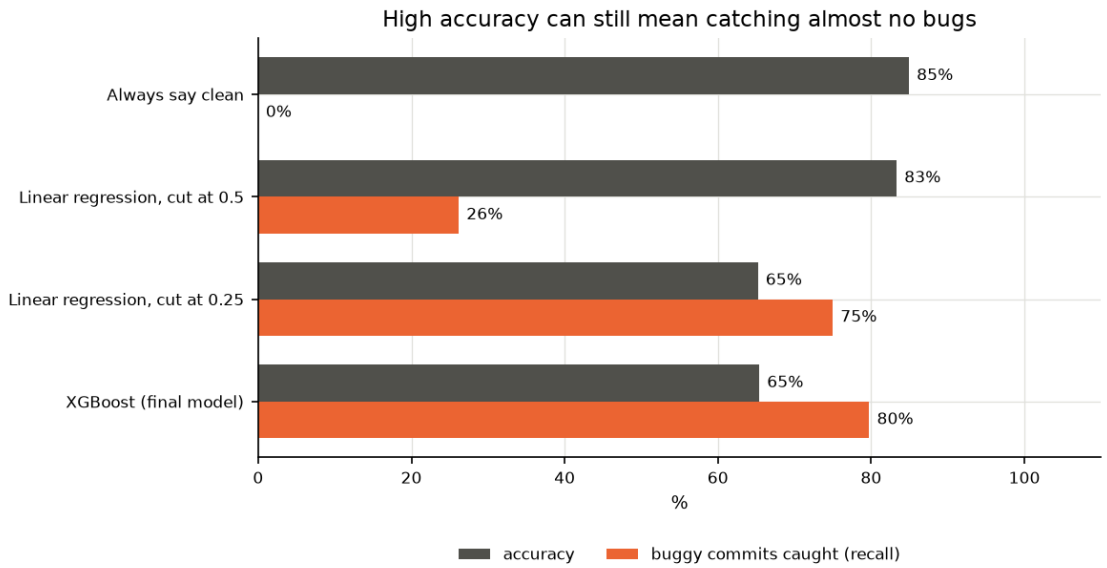
A reviewer who checks only one commit in five finds about half the bugs. That is 2.4 times better than picking at random.

Confusion matrix on the 558 test commits

	flagged	not flagged
buggy	67	17
clean	176	298

It flags 44% of commits at this threshold. A team would tune it to their review budget.

A simple baseline, and a learning reviewer



Linear regression: saying "clean" for every commit is 85% accurate and catches no bugs. Accuracy hides what matters.

Supervised vs reinforcement learning

Our models learn from answers we already know. A reinforcement learner is never shown the answers. It tries, gets a reward, and adjusts.

A reviewer that learns from feedback

859

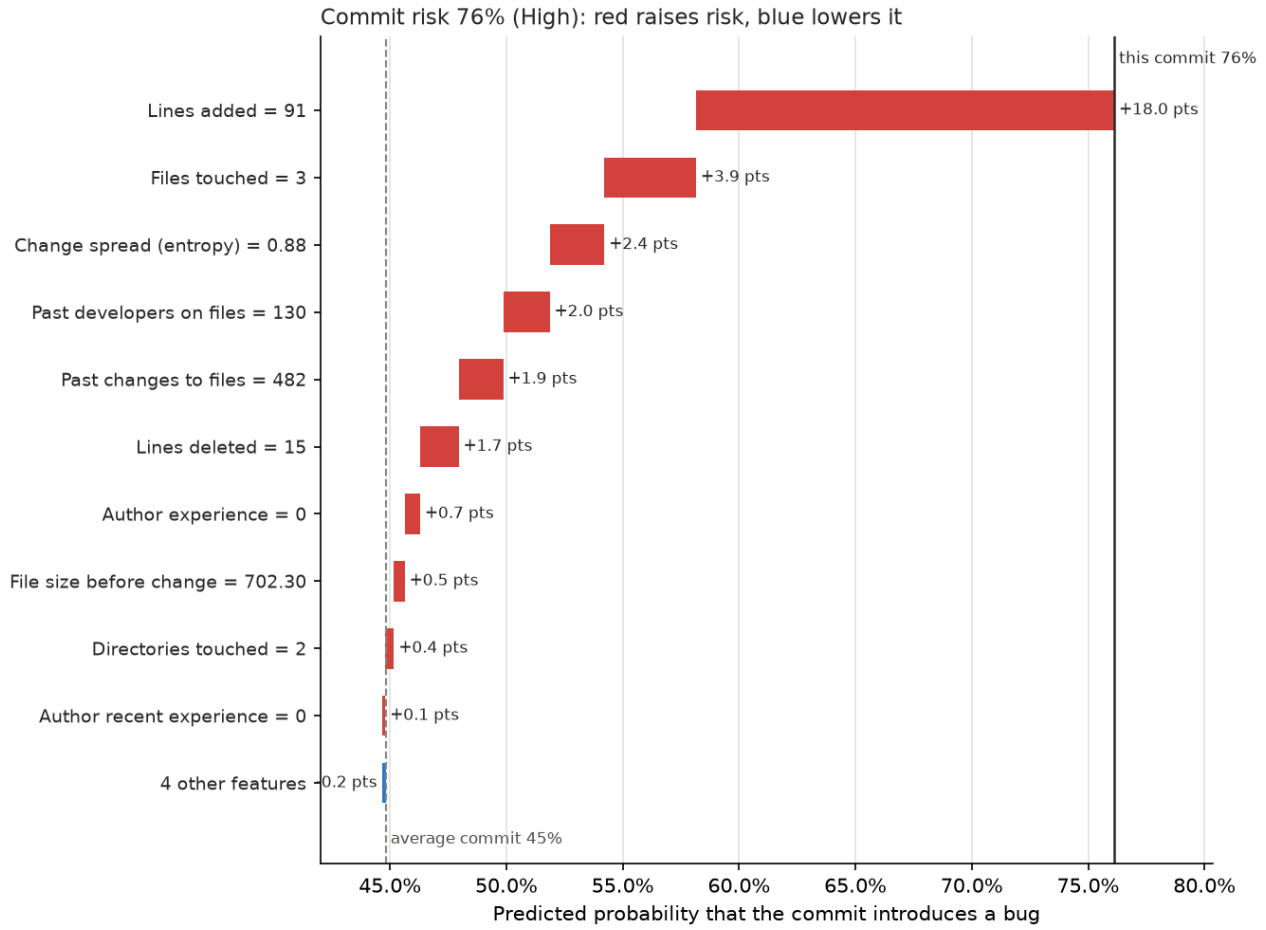
bugs found while learning

433

picking at random

Out of 1,500 reviews. It learned to focus on the biggest commits.

Why is this commit risky?



SHAP splits one prediction into a sum. The average risk (44.8%) plus one contribution per feature adds up exactly to this commit's risk (76.1%).

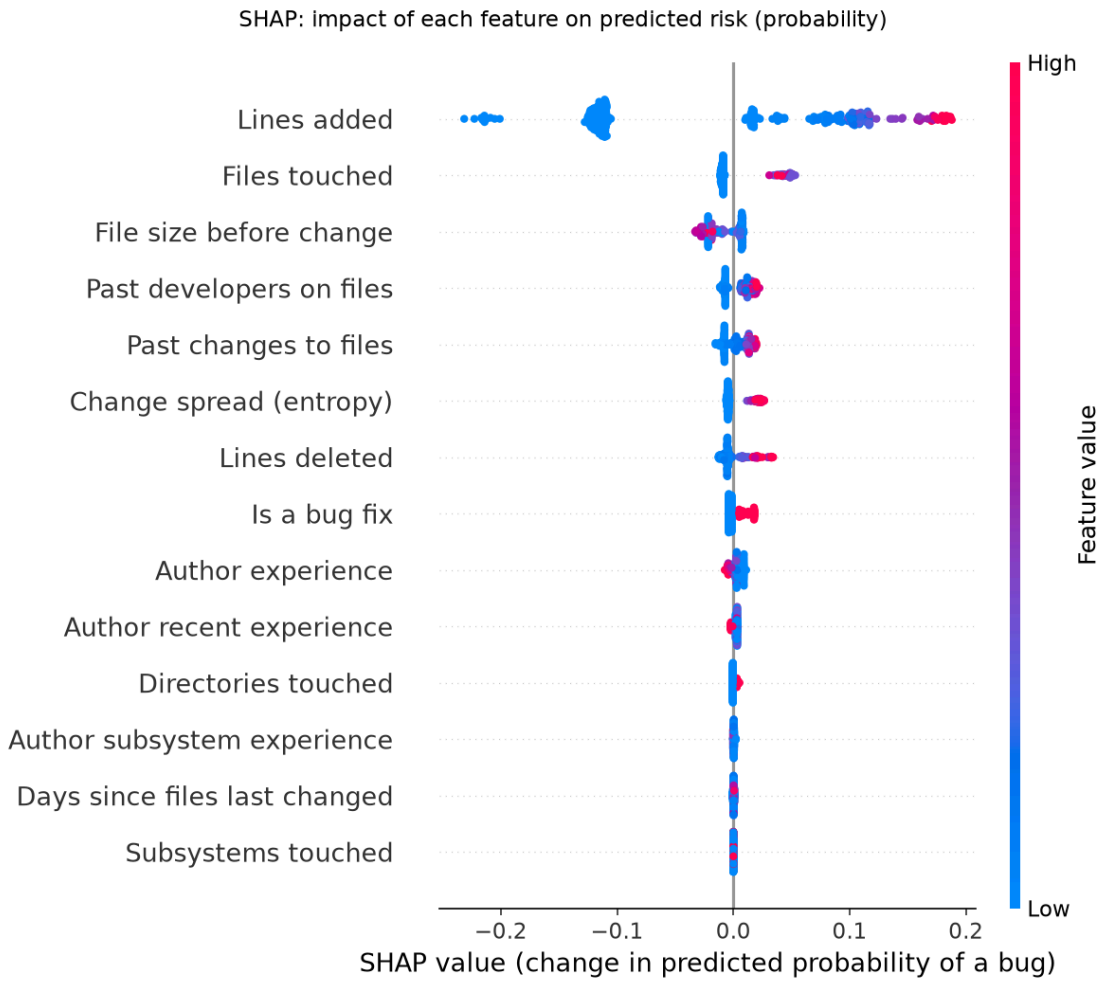
> "Adds 91 lines, more than 96% of past commits, which raises the risk by 18.0 percentage points."

We compare each value with past commits so the sentence means something to a developer.

LIME, our second method, agreed on 60% of the top three features.

Lundberg and Lee (2017), Ribeiro et al. (2016)

What drives risk overall



1. **Lines added dominates.** It moves the risk by about 11 points on average, more than everything else combined.
2. Then files touched, how many people worked on those files before, and how spread out the change is.
3. Experienced authors and larger existing files make a commit slightly safer.
4. Every direction makes sense, which suggests the model learned real patterns and not noise.

Accuracy isn't the whole story

Logistic regression, during development

"Modifies 3 files, which lowers the risk."

Files touched and entropy have a correlation of 0.97, so the model gave them large weights with opposite signs that cancel out.

0.725 CV ROC-AUC

0.769 test ROC-AUC

Tuned XGBoost, the final model

"Modifies 3 files, more than 94% of past commits, which raises the risk."

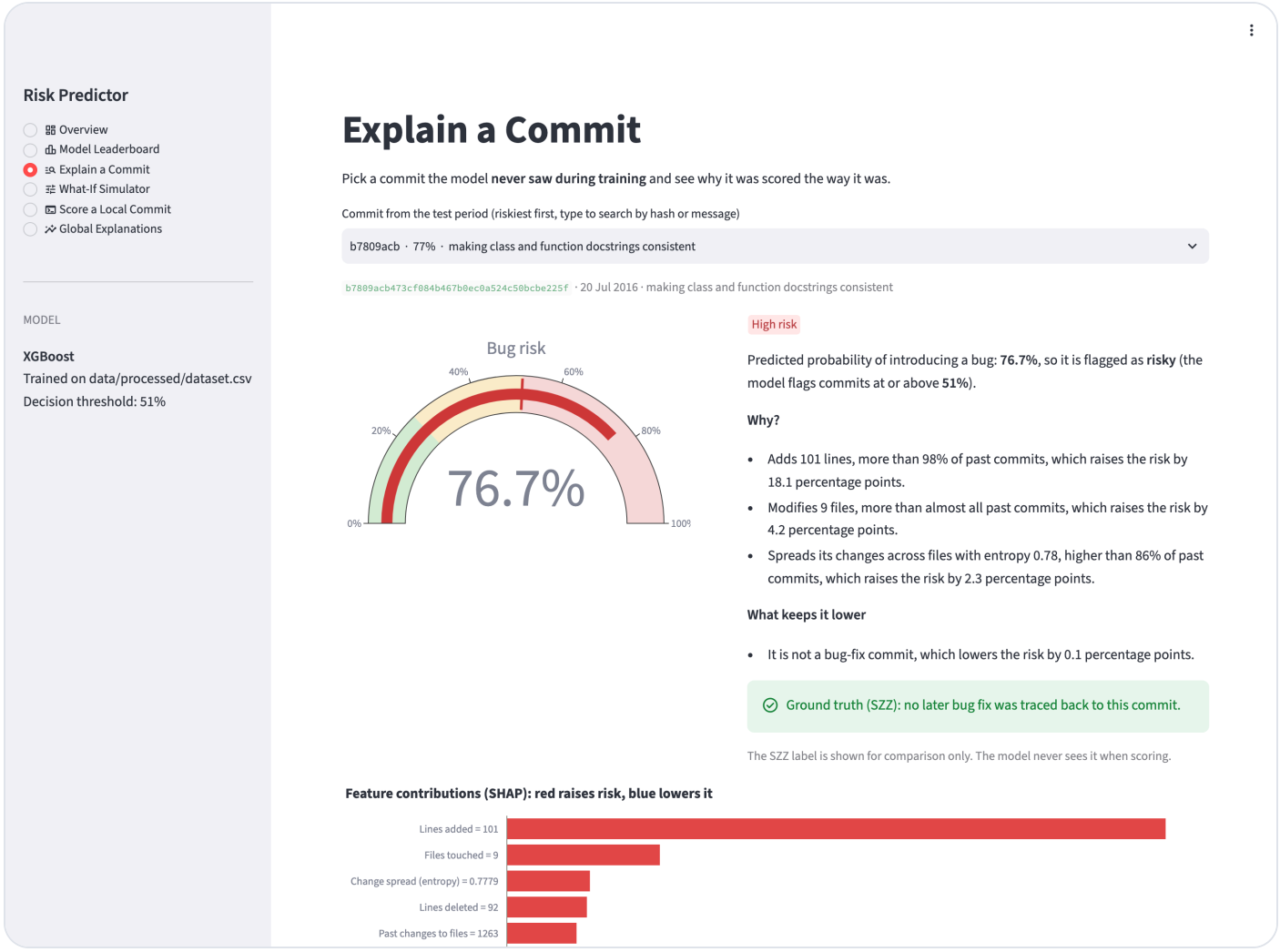
Shallow trees with column sampling don't learn these reversals.

0.732 CV ROC-AUC

0.779 test ROC-AUC

The scores are almost the same, but only one model gave explanations a reviewer would trust. Checking that explanations make sense is part of evaluating a model.

The app



1. **Overview** of the data and the bug rate over time
2. **Leaderboard** of all 13 models
3. **Explain a commit** with a risk gauge, reasons and SHAP
4. **What-if simulator** that shows how splitting a commit changes its risk
5. **Score a local commit** from any git repo
6. **Global explanations**

Built with Streamlit. The same scoring also works from the command line.

Try it online:
`code-change-risk-predictor.streamlit.app`

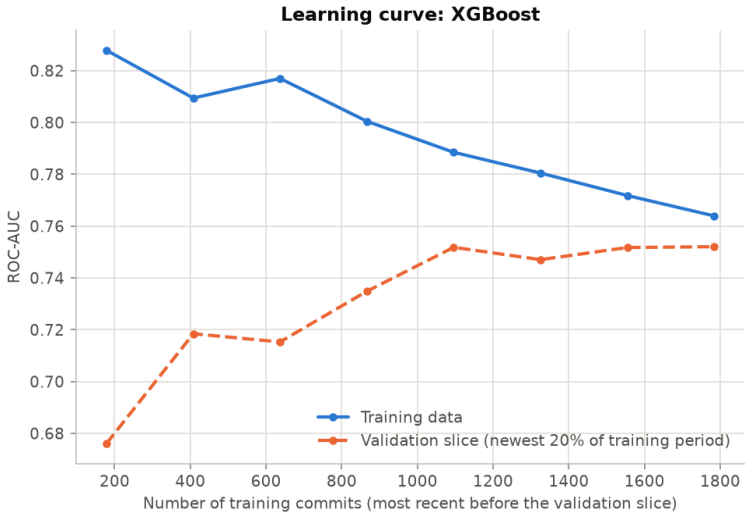
Limitations and next steps

Limitations

- SZZ labels are noisy
- Data from one project only
- The bug rate drifts over time, from 23.6% to 15.1%
- Class weights shift the probabilities up, so read them as relative risk

Next steps

- Train on one project, test on another
- A cleaner SZZ variant that skips refactorings
- Features from the diff content itself
- Incremental scoring, so a git hook answers in under a second



The learning curve flattens after about 1,100 commits, so more data from the same project won't help much. Better features or cleaner labels will.

Thank you

Questions are welcome.

Code and report: `github.com/AKSHATSPAR/code-change-risk-predictor`

Live app: `code-change-risk-predictor.streamlit.app`